Optimization

May 19, 2026

1 Introduction

Fundamentally, calibration is an *optimization* problem, in the sense that we are given some data, have a model of the data, and want to find the model parameters that best fit the data. To that end, I would like you to work through some progressively more challenging optimization problems to build up to gaining a rough understanding of how CorrCal works. For this notebook, you will only need `numpy`, `matplotlib`, and `scipy` installed.

Before jumping into any of the details, I want to take some time to more clearly define what I mean when I discuss optimization problems. The simple setup is that I have some data $\{d_i\}$, I have some parametric model of the data $m(\vec{\theta})$, and I want to find the set of parameters $\{\theta_j\}$ that gives me the "best fit" to the data. (A bit more generally, the model doesn't even need to be parametric, but we won't get into that here.) Mathematically, I can define some notion of distance between the model and the data, and I can find the best-fit parameters by minimizing the "distance" between the model and the data. So, the optimization problem we're concerned with here is as follows:

Given some objective function $f(\vec{\theta}, \vec{d})$, what is the set of parameters $\{\hat{\theta}_j\}$ that minimizes f?

I like to think that there are different "classes" of optimization problems that are characterized by the choice of objective function, and these classes are split into "subclasses" based on the techniques employed to find the solution.

1.1 Imports

[1]:
```
import numpy as np

import matplotlib.pyplot as plt
from scipy.optimize import minimize
%matplotlib inline
```

2 Least-Squares

In the language introduced above, the method of least-squares would qualify as one class of optimization problems. The objective function used in least-squares approach is the (hopefully familiar) χ^2:

$$\chi^2 = \sum_i \frac{|d_i - m_i(\vec{\theta})|^2}{\sigma_i^2}.$$

In words, χ^2 is the inverse-variance weighted average of the distance between the data and the model (under the L^2, or Euclidean, norm). The variance σ_i^2 is just a measure of how noisy the data is, with noisier data having higher variance. Inverse-variance weighting is therefore a way to encode the idea that we should have more trust in data that is less noisy. There is a natural way of coming to use χ^2 as the objective function, which we will come back to in the Maximum Likelihood section, but for now you can think of χ^2 as a measure of the average distance between the model and the data.

There are basically two flavors of least-squares problems, and which flavor you need to employ depends on the nature of the model, which we'll explore further in the following section. (The Wikipedia page reveals that there is more complexity than this, but I've never needed to know more than a slightly more generalized form of what is covered below.)

2.1 Linear Least-Squares

Linear least-squares refers to the class of problems where the model $\vec{m}$ is a linear transformation of the model parameters $\vec{\theta}$, in the sense that we may write

$$\vec{m} = \mathbf{A}\vec{\theta},$$

where the *design matrix* $\mathbf{A}$ encodes that linear transformation. Making this substitution, χ^2 can be written as a simple matrix equation:

$$\chi^2 = (\vec{d} - \mathbf{A}\vec{\theta})^T \mathbf{N}^{-1} (\vec{d} - \mathbf{A}\vec{\theta}),$$

where the noise matrix $\mathbf{N}$ is a diagonal matrix whose entries are the variance in the data $N_{ij} = \sigma_i^2 \delta_{ij}$. Remember that the goal is to find the choice of model parameters $\vec{\theta}$ that produces the closest match to the observed data $\vec{d}$, which we can do by minimizing χ^2. Since χ^2 is non-negative and a quadratic function of the model parameters, we can find the best-fit parameters $\widehat{\theta}$ that minimize χ^2 by determining where the gradient of χ^2 vanishes. That is, the optimal solution is the one satisfying

$$\frac{\partial \chi^2}{\partial \vec{\theta}} = 0.$$

Skipping over some of the math, this gives us the condition

$$\mathbf{A}^T \mathbf{N}^{-1} \vec{d} - \mathbf{A}^T \mathbf{N}^{-1} \mathbf{A} \vec{\theta} = 0,$$

which we can invert to obtain the best-fit parameters

$$\widehat{\theta} = (\mathbf{A}^T \mathbf{N}^{-1} \mathbf{A})^{-1} \mathbf{A}^T \mathbf{N}^{-1} \vec{d}.$$

Solving linear least-squares problems thus boils down to a problem of constructing the design matrix and estimating the noise in the data. Let's work through a few examples.

2.1.1 Fitting a Polynomial

For this first example, we will generate some noisy data that follows a polynomial, then perform a linear least-squares fit to determine the coefficients of the polynomial and check against the true values used in simulation to validate our solution. This last step is not a luxury we have when working with real data, but it is an important step to perform when proposing a method of fitting the data (in other words, it is a very good idea to *validate* that your proposed optimization routine works).

To make the math a bit clearer, our data will be constructed via

$$y_i = \sum_m a_m x_i^m + n_i,$$

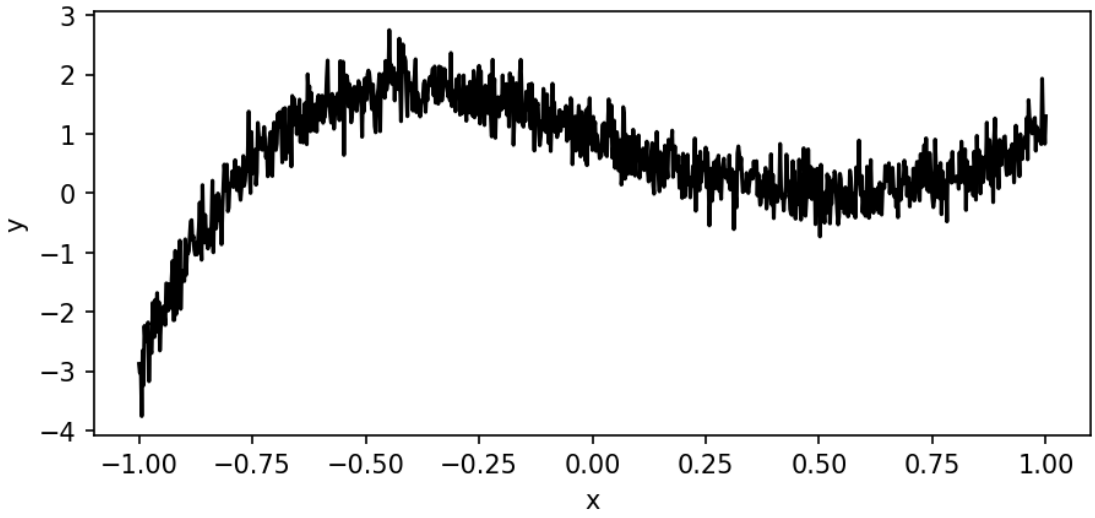
where $\{a_m\}$ are the model parameters and n_i is the noise in measurement i. We will use mean-zero Gaussian random noise with a fixed variance, which is mathematically encoded in the expression

$$n_i \sim \mathcal{N}(0, \sigma^2).$$

[2]:
```
# Define the polynomial coefficients.
poly_coeffs = np.array([1, -3, 0, 5, -2])

# Come up with a range of x-values for which we will simulate data.
x_values = np.linspace(-1, 1, 1001)
print(len(x_values))
# Now generate the simulated data. There is a more clever way to do this,
# but I think this way is a bit easier to understand.
y_values = np.zeros_like(x_values)
for m, a_m in enumerate(poly_coeffs):
    y_values += a_m * x_values**m

# Now add noise.
var = 0.1
y_values += np.random.normal(size=y_values.size, loc=0, scale=np.sqrt(var))
```

```
1001
```

[3]:
```
# Let's take a quick look at the simulated data to see if it looks reasonable.
plt.figure(figsize=(7,3), dpi=150)
plt.plot(x_values, y_values, color="k")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



This quick check confirms that the simulated data does indeed look like a polynomial with some additive noise.

[4]:
```
# First step, we need to define the design matrix.
# Thinking about shapes (how are matrix products defined?):
# There must be as many columns as there are model parameters.
# There must be as many rows as there are data points.
A = np.zeros((x_values.size, poly_coeffs.size), dtype=float)
print(A)
for m in range(poly_coeffs.size):
    A[:,m] = x_values ** m

# Construct the inverse noise matrix, but we only need the diagonal.
N_inv = np.ones(x_values.size) / var

# Now we can figure out the best-fit values in one line!
fit_coeffs = np.linalg.inv(
    A.T @ (N_inv[:,np.newaxis] * A)
) @ A.T @ (N_inv * y_values)
```

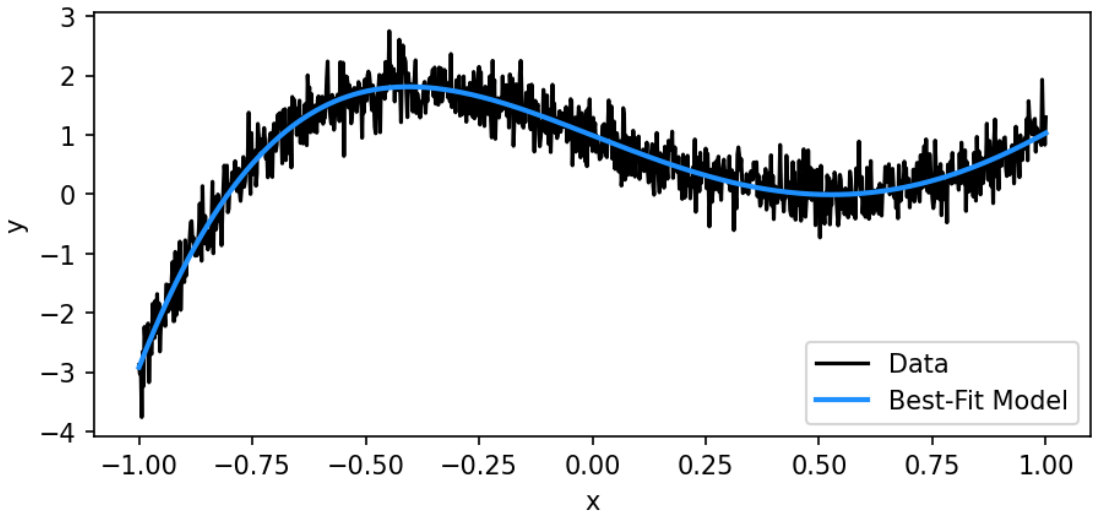
```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 …
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
```

[5]:
```
# Let's see if the fit values match what we put in!
print(f"Input parameters: {poly_coeffs}")
print(f"Best-fit parameters: {fit_coeffs}")
```

```
Input parameters: [ 1 -3  0  5 -2]
Best-fit parameters: [ 0.99138672 -2.97509589 -0.0333702   4.94950856
-1.90098256]
```

That looks pretty close to me! Let's evaluate the model and compare it to the input data.

[6]:
```
# In case you are unfamiliar with the @ operator, "@" used here stands for␣
  ↪"mATrix multiply"
model = A @ fit_coeffs
plt.figure(figsize=(7,3), dpi=150)
plt.plot(x_values, y_values, color="k", label="Data")
plt.plot(x_values, model, color="dodgerblue", lw=2, label="Best-Fit Model")
plt.legend()
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



This looks like a pretty good fit to me! We can make a more quantitative estimate of how good the fit is by computing χ^2 *per degree of freedom*, or χ^2/DoF. Unfortunately, "degree of freedom" is used in kind of a backwards way when people are talking about model fitting compared to how it is used in computing χ^2/DoF—colloquially, when someone is talking about "adding degrees of freedom" (or "opening up degrees of freedom") to (in) a model, they mean adding extra model parameters, but when computing χ^2/DoF, the "degrees of freedom" is actually the number of extra data points you have beyond the number of model parameters. A good rule-of-thumb (which comes from figuring out, on average, what should I obtain for χ^2/DoF if my fit is good?) is that a good fit and properly estimated errors will have χ^2/DoF ≈ 1. You may have already been introduced to

this quantity as "reduced χ^2" in some of your physics labs.

[7]:
```
chisq = (y_values - model) @ (N_inv * (y_values - model))
dof = y_values.size - poly_coeffs.size
chisq_per_dof = chisq / dof
print(chisq_per_dof)
```

```
0.9830615922668556
```

Something that was likely not taught in your physics labs, however, is how close χ^2/DoF should be to 1 for the fit to be considered "good". One way to think about this is to treat χ^2 as a random variable that follows some probability distribution. Perhaps unsurprisingly, χ^2 follows a chi-squared distribution, which is uniquely characterized by the degrees of freedom. A chi-squared distribution with DoF degrees of freedom has mean DoF and variance 2DoF, which means χ^2/DoF has mean 1 and variance 2/DoF. More degrees of freedom therefore corresponds to a tighter distribution of χ^2/DoF values centered on 1. Intuitively, the more data we use to optimize our fit (i.e., the more degrees of freedom in the statistical sense), the closer we ought to expect χ^2/DoF to be 1 to consider our fit as "good". With this framework in mind, we can ask how many standard deviations away from 1 our measured χ^2/DoF is, and if this number is high, then our fit is probably not good.

[8]:
```
chisq_per_dof_stdev = np.sqrt(2 / dof)
(1 - chisq_per_dof) / chisq_per_dof_stdev
```

[8]:
```
np.float64(0.377996043569313)
```

As a rule of thumb, if our fit has χ^2/DoF within one or two standard deviations of 1, then it is likely that our model is a good description of the data and we have correctly estimated the noise level in the data. Otherwise, it may be time to re-evaluate either the model or our approach for estimating the noise in the data.

2.1.2 Fitting a Power Law

In astronomy, we often need to work with power laws, so it's useful to learn how to fit them. A power law is something that looks like the following:

$$S(\nu) = S_0(\nu/\nu_0)^{-\beta}.$$

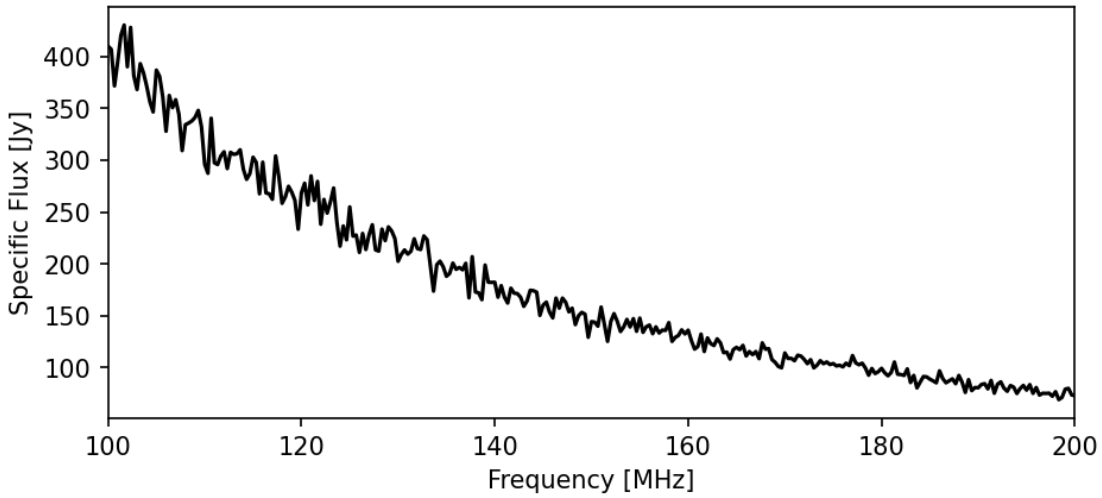
In this case, S_0 is the flux that is observed at frequency ν_0, and β is called the *spectral index*.

Question:
* Take the logarithm of the above quation to linearize the problem. How are the linearized parameters related to the reference flux S_0, the reference frequency ν_0, and the spectral index β?

Let's cook up some data and try to fit a power law to it.

[124]:
```
# First, simulate the "true" data.
spectral_index = 2.5
ref_flux_Jy = 150
ref_freq_MHz = 150
obs_freqs_MHz = np.linspace(100, 200, 301)
```

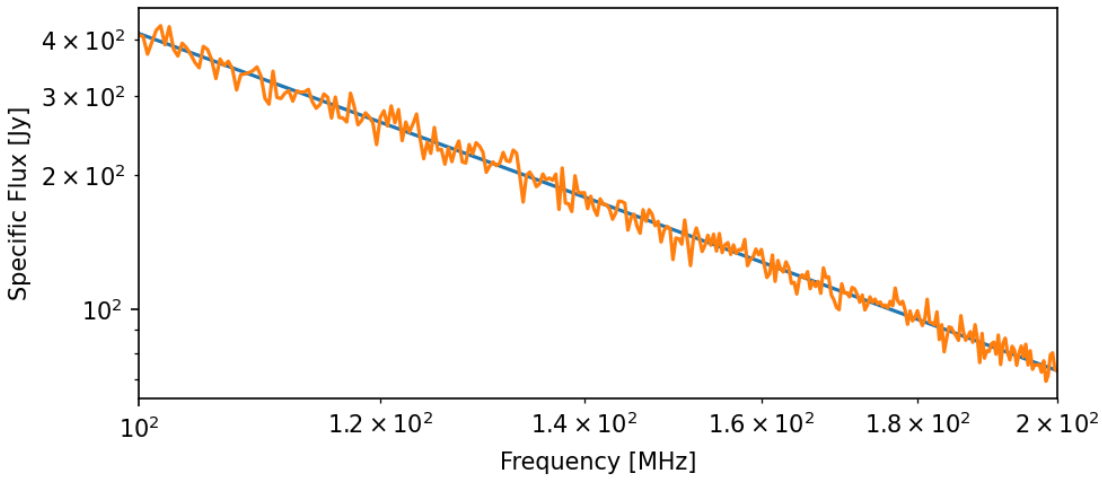
```
true_data = ref_flux_Jy * (obs_freqs_MHz/ref_freq_MHz) ** -spectral_index
# Now add some noise that scales with the brightness of the signal.
noisy_data = true_data * (
    1 + np.random.normal(size=true_data.size, loc=0, scale=0.05)
)

# Then take a look at the data.
fig, ax = plt.subplots(1, 1, figsize=(7,3), dpi=150)
ax.set_xlabel("Frequency [MHz]")
ax.set_ylabel("Specific Flux [Jy]")
ax.plot(obs_freqs_MHz, noisy_data, color="k")
ax.set_xlim(obs_freqs_MHz[0], obs_freqs_MHz[-1])
plt.show()
```



Does this look right? Try making a log-log plot and see if that looks the way you expect based on your answer to the question at the beginning of this section.

[125]:
```
# Write your plotting code here.
"""Does this look right?
Try making a log-log plot and see if that looks the way you expect based on␣
 ↪your answer to the question at the beginning of this section.
After plotting the log-log plot, the result is what we are expected after␣
 ↪linearizing the power law into y=a+bx format where y= log() a =, b=-beta"""
fig, ax = plt.subplots(1, 1, figsize=(7,3), dpi=150)
ax.set_xlabel("Frequency [MHz]")
ax.set_ylabel("Specific Flux [Jy]")
plt.loglog(obs_freqs_MHz, true_data)
plt.loglog(obs_freqs_MHz, noisy_data)
ax.set_xlim(obs_freqs_MHz[0], obs_freqs_MHz[-1])
plt.savefig("power_law_loglog.pdf", format="pdf", bbox_inches="tight")
```

```
plt.show()
```



Now let's suppose we only know the observed frequencies and the measured data, then try to fit a power law to it. I'll outline the steps, but you'll need to fill in the code. For this problem, we'll ignore the inverse-variance weighting (i.e., we will use *uniform* weighting).

Questions:
* If we're not inverse variance weighting, what is the correct expression for the noise matrix **N**? * What is the expression for the best-fit parameters in the case of uniform weighting? * How many rows and columns should the design matrix **A** have for this problem? What should the terms in the design matrix be?

[ ]:
```
"""
Quesiton:
If we're not inverse variance weighting, what is the correct expression for the␣
 ↪noise matrix ?
Uniform weighting is when wi = 1 (for X^2 N^-1 is 1/varinace^2), thus for␣
 ↪uniform weighting, the noise matrix is N_inv= I (identity matrix)
What is the expression for the best-fit parameters in the case of uniform␣
 ↪weighting? theta = np.linalg.inv(A.T@A)@A.T@d since N_inv is an identity␣
 ↪matrix
How many rows and columns should the design matrix A have for this problem?␣
 ↪What should the terms in the design matrix be?
the column of the A matrix should be len(obs_frequency), the row should be
"""
```

[126]:
```
# Define the design matrix.
# Linearize the variables
x = np.log(obs_freqs_MHz)
y = np.log(noisy_data)
# Build design matrix
```

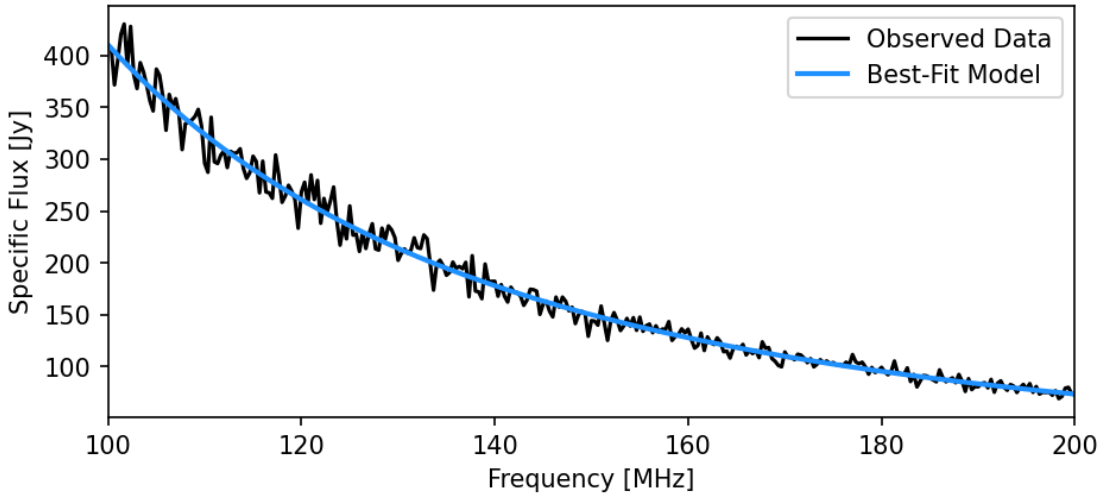
```
A = np.zeros((x.size, 2))
A[:,0] = 1
A[:,1] = x
print(len(A))
# Compute the best-fit parameters.
best_fit = np.linalg.inv(A.T @ A) @ A.T @ y
print("best_fit size",best_fit.size)
a = best_fit[0]
b = best_fit[1]
print(best_fit)
# Convert these to the spectral index and the flux at 150 MHz.
fit_spectral_index = -b #beta
fit_ref_flux = np.exp(a - fit_spectral_index * np.log(ref_freq_MHz)) #So
# Compare the result against the true values.
print("True spectral index:", spectral_index)
print("Fitted spectral index:", fit_spectral_index)
print("True reference flux:", ref_flux_Jy)
print("Fitted reference flux:", fit_ref_flux)
```

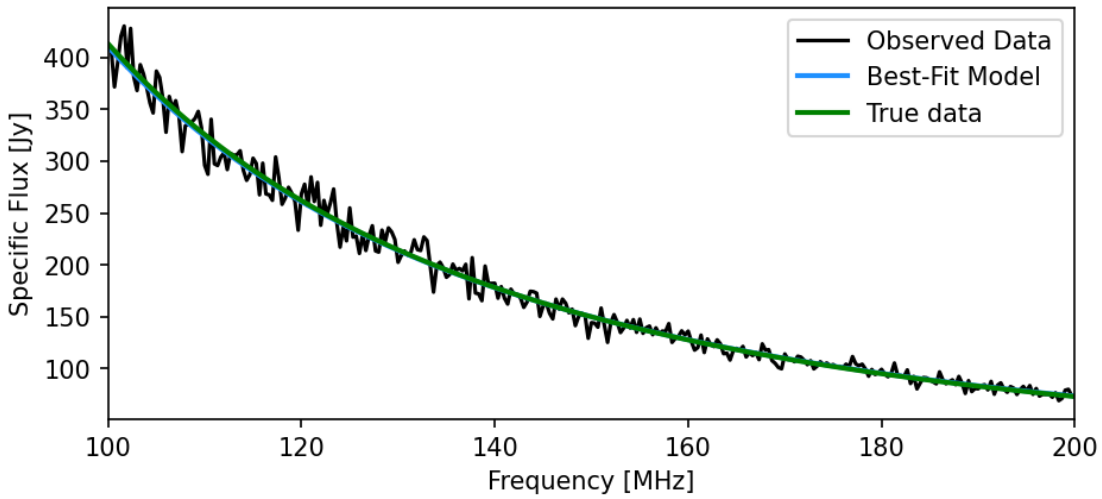
```
301
best_fit size 2
[17.44309122 -2.48115565]
True spectral index: 2.5
Fitted spectral index: 2.4811556494080644
True reference flux: 150
Fitted reference flux: 150.04348466541273
```

How do your best-fit values compare against the true values? Try making a plot of the best-fit *model* against the data.

[127]:
```
# Evaluate the best-fit model.
fit_model = fit_ref_flux * (obs_freqs_MHz / ref_freq_MHz) **␣
 ↪(-fit_spectral_index) #plug the best fit parameter back into the power law␣
 ↪to estimate the outcome
# Plot the best-fit model and the data.
plt.figure(figsize=(7,3), dpi=150)
plt.plot(obs_freqs_MHz, noisy_data, color="k", label="Observed Data")
plt.plot(obs_freqs_MHz, fit_model, color="dodgerblue", lw=2, label="Best-Fit␣
 ↪Model")
plt.legend()
plt.xlabel("Frequency [MHz]")
plt.ylabel("Specific Flux [Jy]")
plt.xlim(obs_freqs_MHz[0], obs_freqs_MHz[-1])
plt.savefig("power_law_fit_only.pdf", format="pdf", bbox_inches="tight")
plt.show()

plt.figure(figsize=(7,3), dpi=150)
plt.plot(obs_freqs_MHz, noisy_data, color="k", label="Observed Data")
```

```
plt.plot(obs_freqs_MHz, fit_model, color="dodgerblue", lw=2, label="Best-Fit␣
 ↪Model")
plt.plot(obs_freqs_MHz, true_data, color="g", lw=2, label="True data")
plt.legend()
plt.xlabel("Frequency [MHz]")
plt.ylabel("Specific Flux [Jy]")
plt.xlim(obs_freqs_MHz[0], obs_freqs_MHz[-1])
plt.savefig("power_law_fit_vs_true.pdf", format="pdf", bbox_inches="tight")
plt.show()
```



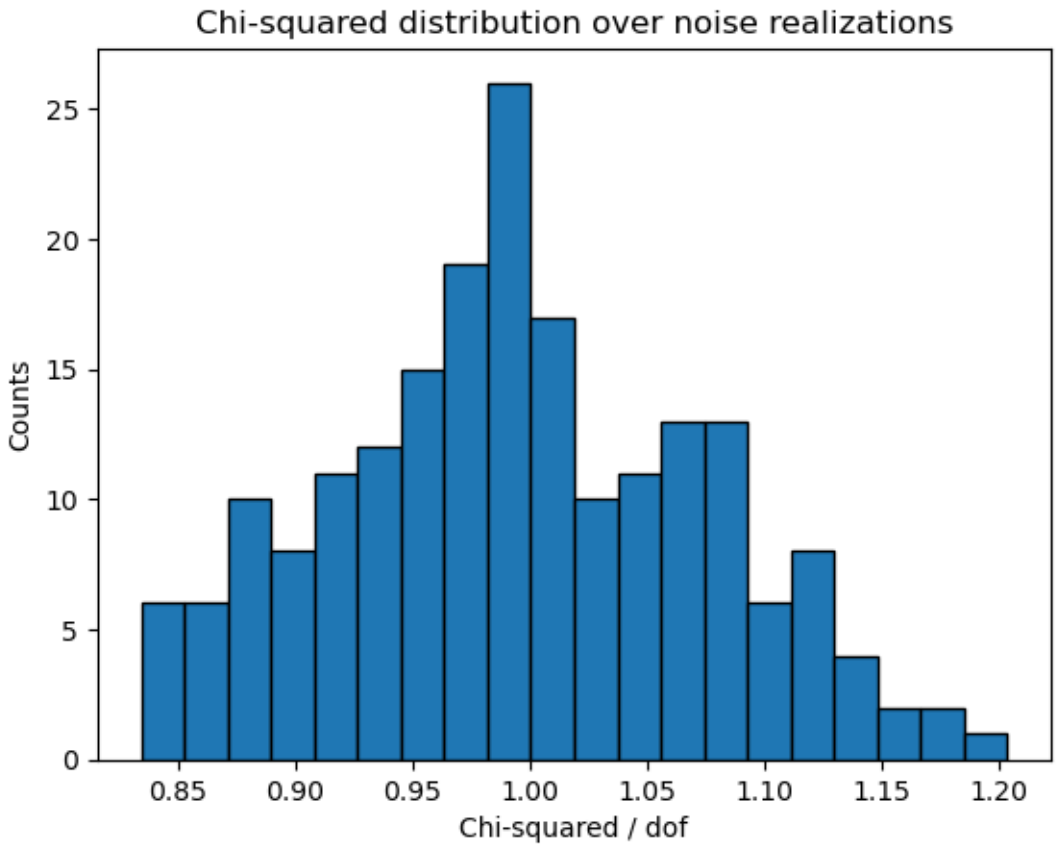


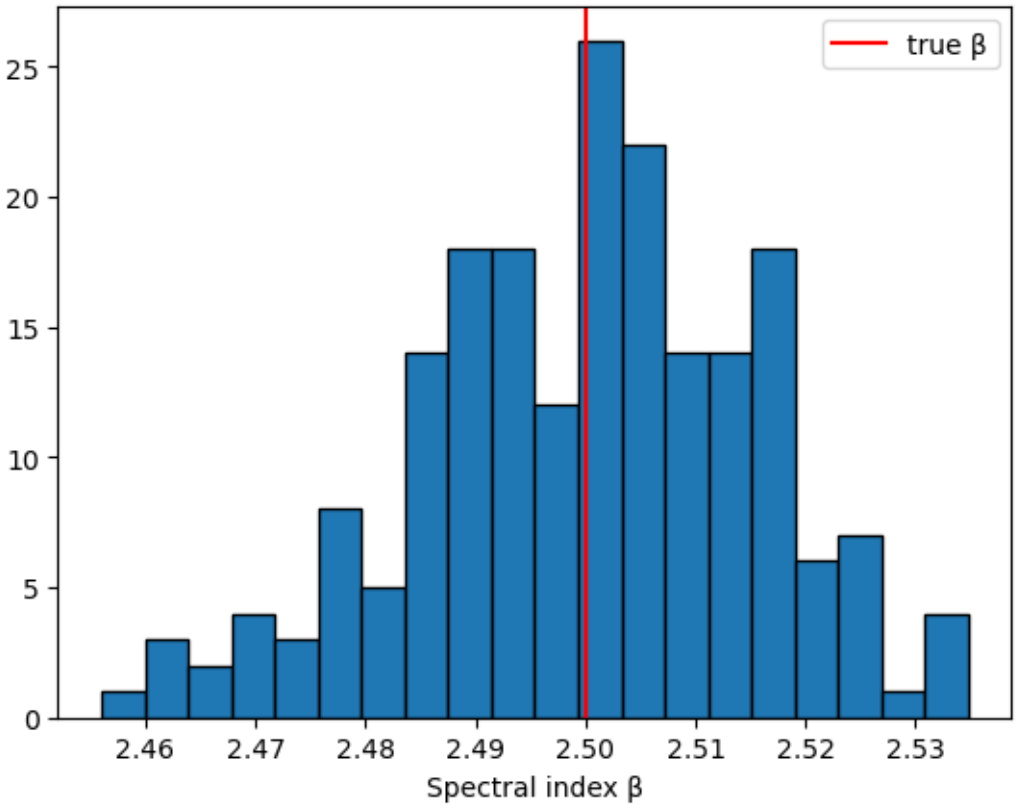
[87]:
```
#checking if the best fit model is valid
fit_model = fit_ref_flux * (obs_freqs_MHz / ref_freq_MHz) **␣
 ↪(-fit_spectral_index)
sigma = 0.05 * true_data   # <-- use SAME fractional noise used in simulation
residuals = noisy_data - fit_model
chisq = np.sum((residuals / sigma) ** 2)
dof = len(noisy_data) - best_fit.size
chisq_per_dof = chisq / dof
chisq_per_dof_stdev = np.sqrt(2 / dof)
z_score = (chisq_per_dof - 1) / chisq_per_dof_stdev
print("Chi-squared:", chisq)
print("DOF:", dof)
print("Reduced chi-squared:", chisq_per_dof)
print("Expected stdev:", chisq_per_dof_stdev)
print("Z-score:", z_score)
```

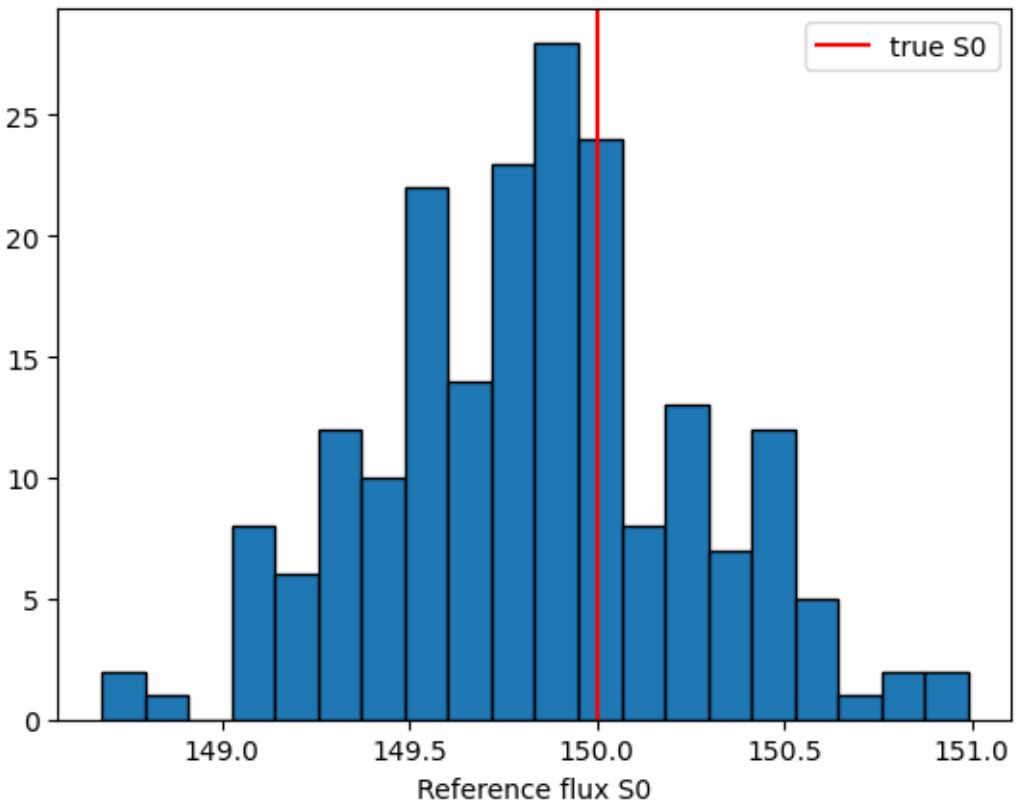
```
Chi-squared: 292.3588130540306
DOF: 299
Reduced chi-squared: 0.9777886724215069
Expected stdev: 0.08178608201095307
Z-score: -0.2715783301065635
```

[91]:
```
def fit_one_realization(obs_freqs_MHz, noisy_data, ref_freq_MHz):
    x = np.log(obs_freqs_MHz / ref_freq_MHz)
    y = np.log(noisy_data)

    A = np.zeros((x.size, 2))
    A[:, 0] = 1
    A[:, 1] = x

    best_fit = np.linalg.inv(A.T @ A) @ A.T @ y
    a = best_fit[0]
    b = best_fit[1]

    s0 = np.exp(a)
    beta = -b

    return s0, beta

N = 200
beta_vals = []
s0_vals = []
chi2_vals = []
for i in range(N):
    # new noise each time
    noisy_data = true_data * (
        1 + np.random.normal(0, 0.05, size=true_data.size)
```

```
    )
    s0_fit, beta_fit = fit_one_realization(
        obs_freqs_MHz,
        noisy_data,
        ref_freq_MHz
    )
    beta_vals.append(beta_fit)
    s0_vals.append(s0_fit)
    # rebuild model for chi-squared
    model = s0_fit * (obs_freqs_MHz / ref_freq_MHz) ** (-beta_fit)
    sigma = 0.05 * true_data  # known noise level
    chi2 = np.sum(((noisy_data - model) / sigma) ** 2)
    dof = len(obs_freqs_MHz) - 2
    chi2_vals.append(chi2 / dof)

plt.hist(chi2_vals, bins=20, edgecolor='k')
plt.xlabel("Chi-squared / dof")
plt.ylabel("Counts")
plt.title("Chi-squared distribution over noise realizations")
plt.show()

plt.hist(beta_vals, bins=20, edgecolor='k')
plt.axvline(spectral_index, color='r', label="true  ")
plt.xlabel("Spectral index  ")
plt.legend()
plt.show()

plt.hist(s0_vals, bins=20, edgecolor='k')
plt.axvline(ref_flux_Jy, color='r', label="true S0")
plt.xlabel("Reference flux S0")
plt.legend()
plt.show()
```

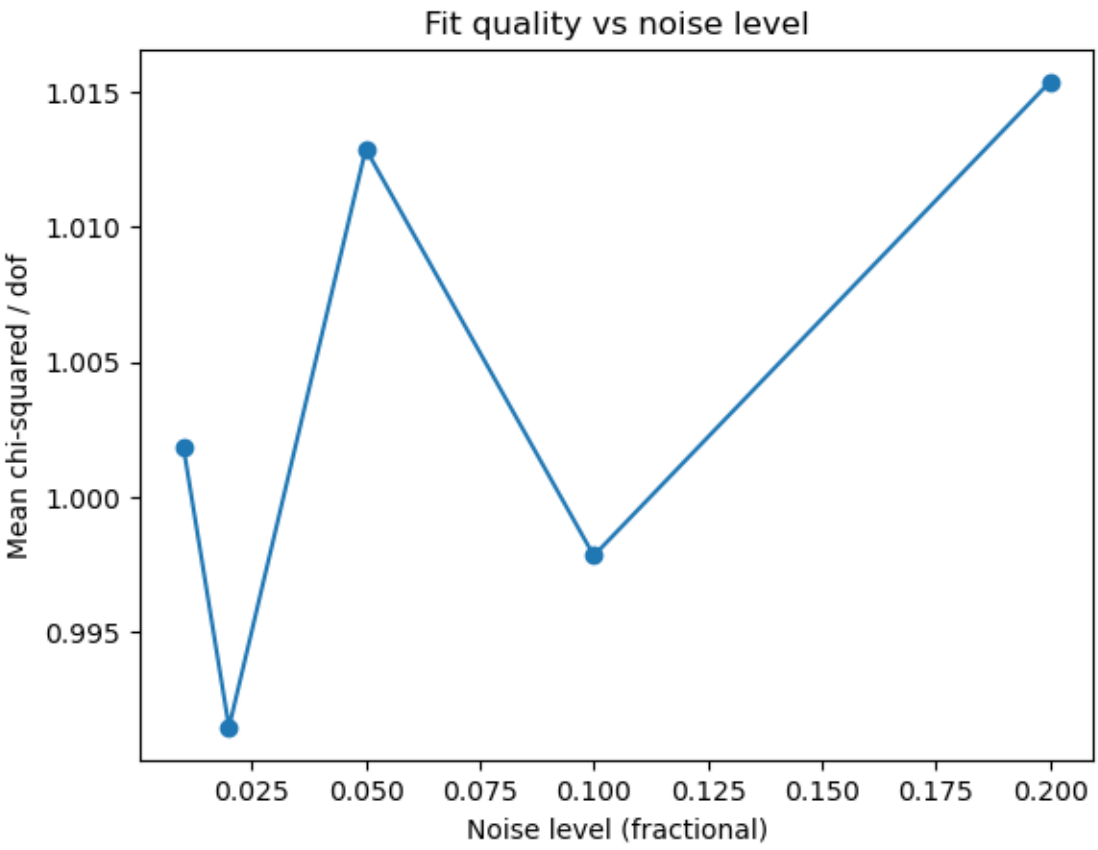

Chi-squared distribution over noise realizations
Counts
0
5
10
15
20
25
0.85
0.90
0.95
1.00
1.05
1.10
1.15
1.20
Chi-squared / dof

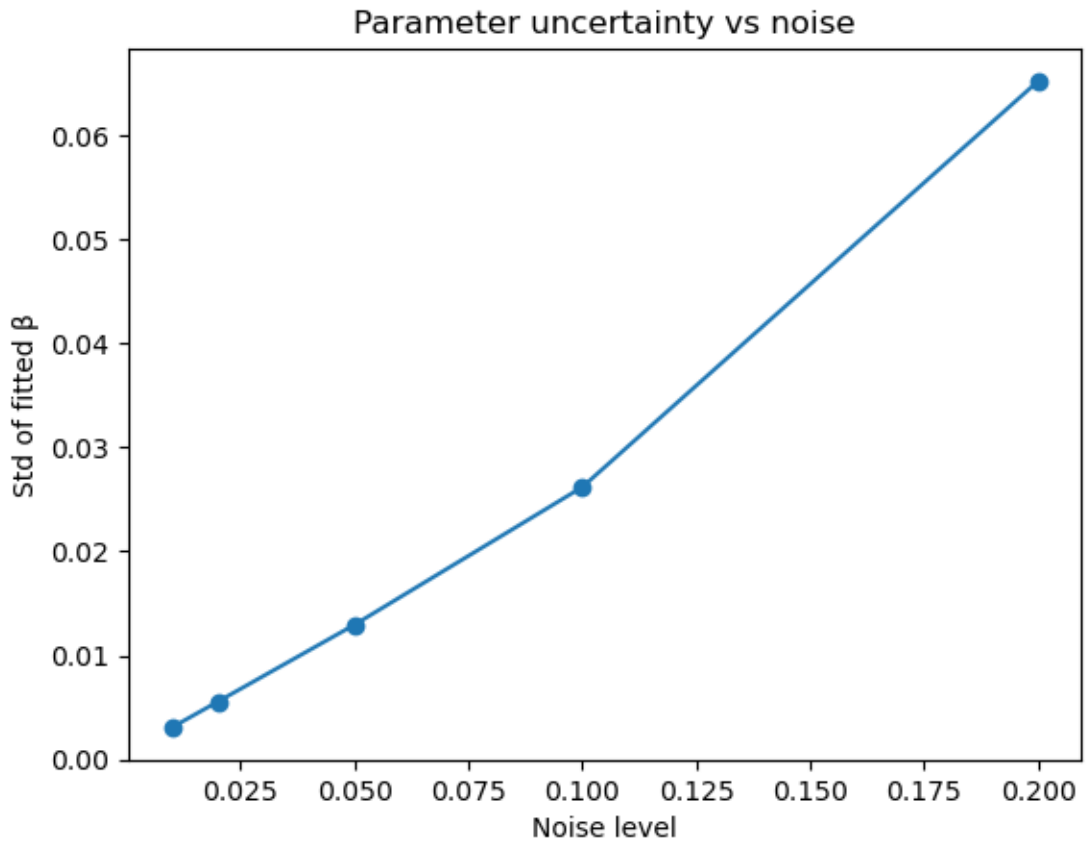

true β
25
20
15
10
5
0
2.46
2.47
2.48
2.49
2.50
2.51
2.52
2.53
Spectral index β



[93]:
```
noise_levels = [0.01, 0.02, 0.05, 0.1, 0.2]
chi2_means = []
beta_spreads = []
for sigma_frac in noise_levels:

    beta_vals = []
    chi2_vals = []

    for i in range(100):  # realizations

        noisy_data = true_data * (
            1 + np.random.normal(0, sigma_frac, size=true_data.size)
        )

        s0_fit, beta_fit = fit_one_realization(
            obs_freqs_MHz,
            noisy_data,
            ref_freq_MHz
        )

        beta_vals.append(beta_fit)
```

```
        model = s0_fit * (obs_freqs_MHz / ref_freq_MHz) ** (-beta_fit)

        sigma = sigma_frac * true_data

        chi2 = np.sum(((noisy_data - model) / sigma) ** 2)
        dof = len(obs_freqs_MHz) - 2

        chi2_vals.append(chi2 / dof)

    chi2_means.append(np.mean(chi2_vals))
    beta_spreads.append(np.std(beta_vals))

plt.plot(noise_levels, chi2_means, marker='o')
plt.xlabel("Noise level (fractional)")
plt.ylabel("Mean chi-squared / dof")
plt.title("Fit quality vs noise level")
plt.show()

plt.plot(noise_levels, beta_spreads, marker='o')
plt.xlabel("Noise level")
plt.ylabel("Std of fitted  ")
plt.title("Parameter uncertainty vs noise")
plt.show()
```

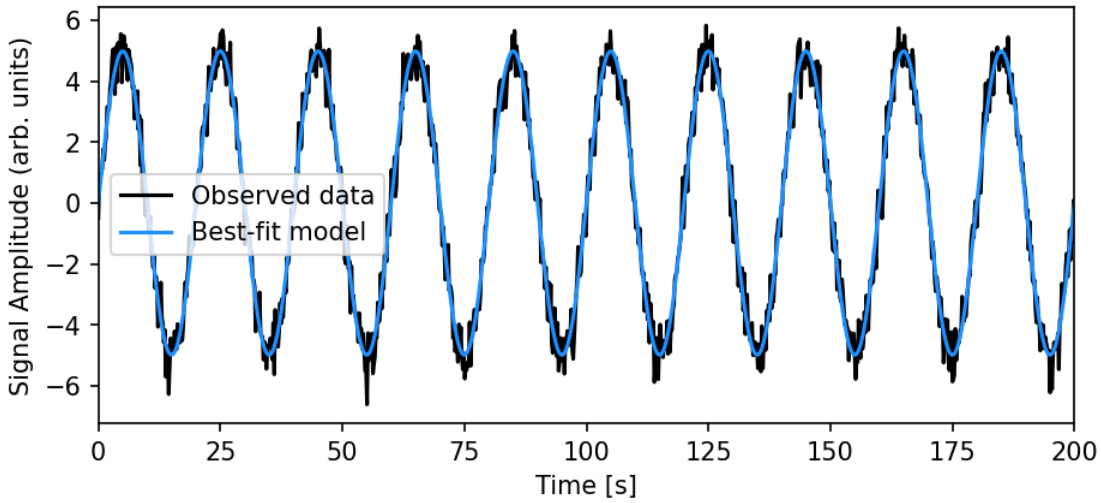

Fit quality vs noise level
Mean chi-squared / dof
1.015
1.010
1.005
1.000
0.995
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
Noise level (fractional)

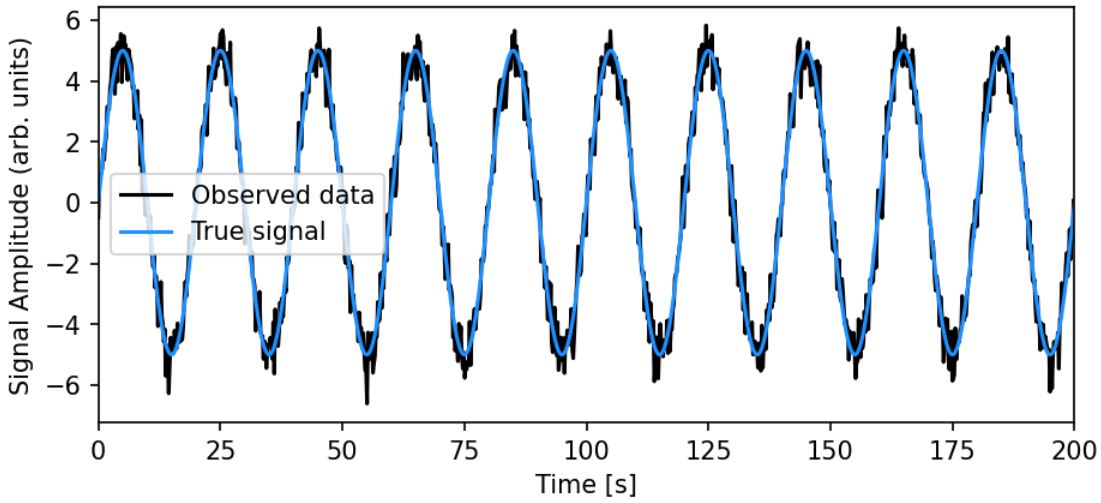


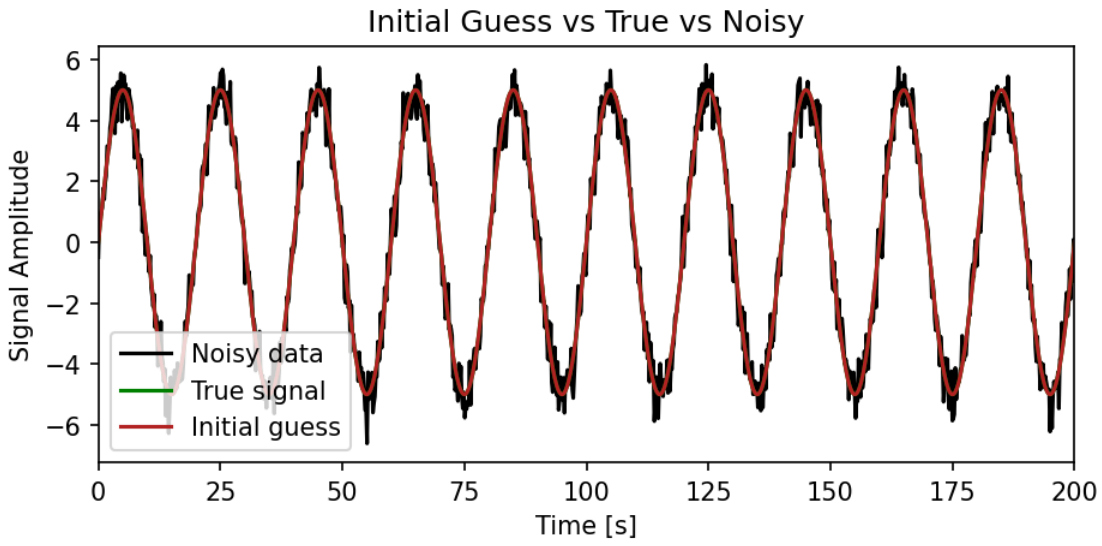
[94]:
```
noise_levels = [0.01, 0.03, 0.05, 0.1, 0.2]
N_realizations = 50
chi2_mean = []
beta_std = []
s0_std = []
x = np.log(obs_freqs_MHz / ref_freq_MHz)
A = np.vstack([np.ones_like(x), x]).T

for sigma_frac in noise_levels:
    beta_vals = []
    s0_vals = []
    chi2_vals = []
    for i in range(N_realizations):
        noise = np.random.normal(0, sigma_frac, size=true_data.size)
        noisy = true_data * (1 + noise)
        y = np.log(noisy)
        a_fit, b_fit = np.linalg.lstsq(A, y, rcond=None)[0]
        beta_fit = -b_fit
        s0_fit = np.exp(a_fit)
        fit_model = s0_fit * (obs_freqs_MHz / ref_freq_MHz) ** (-beta_fit)
```

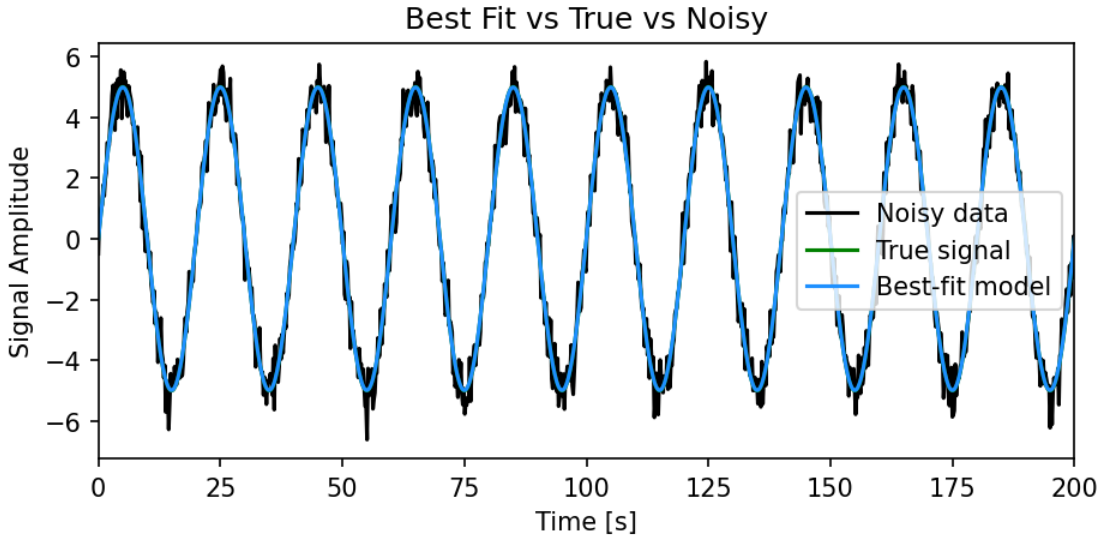
```
        y_data = np.log(noisy)
        y_model = np.log(fit_model)
        chi2 = np.sum(((y_data - y_model) / sigma_frac) ** 2)
        dof = len(obs_freqs_MHz) - 2
        chi2_vals.append(chi2 / dof)
        beta_vals.append(beta_fit)
        s0_vals.append(s0_fit)
    chi2_mean.append(np.mean(chi2_vals))
    beta_std.append(np.std(beta_vals))
    s0_std.append(np.std(s0_vals))
    noisy = np.random.normal(0, sigma_frac, size=true_data.size)
    noisy = true_data * (1 + noise)
    y = np.log(noisy)
    a_fit, b_fit = np.linalg.lstsq(A, y, rcond=None)[0]
    beta_fit = -b_fit
    s0_fit = np.exp(a_fit)
    fit_model = s0_fit * (obs_freqs_MHz / ref_freq_MHz) ** (-beta_fit)
    plt.figure(figsize=(6, 3))
    plt.plot(obs_freqs_MHz, true_data, label="True", linewidth=2)
    plt.plot(obs_freqs_MHz, noisy, label="Noisy", alpha=0.5)
    plt.plot(obs_freqs_MHz, fit_model, label="Fit", linewidth=2)
    plt.title(f"Noise level = {sigma_frac}")
    plt.xlabel("Frequency [MHz]")
    plt.ylabel("Flux [Jy]")
    plt.legend()
    plt.show()

plt.figure(figsize=(6, 3))
plt.plot(noise_levels, chi2_mean, marker='o')
plt.xlabel("Noise amplitude (fractional)")
plt.ylabel("Mean reduced chi-squared")
plt.title("Fit quality vs noise level")
plt.show()

plt.figure(figsize=(6, 3))
plt.plot(noise_levels, beta_std, marker='o')
plt.xlabel("Noise amplitude")
plt.ylabel("Std of fitted spectral index ")
plt.title("Parameter uncertainty vs noise")
plt.show()

plt.figure(figsize=(6, 3))
plt.plot(noise_levels, s0_std, marker='o')
plt.xlabel("Noise amplitude")
plt.ylabel("Std of fitted S0")
plt.title("Flux uncertainty vs noise")
plt.show()
```

```
ax.set_ylabel("Signal Amplitude")
ax.set_title("Initial Guess vs True vs Noisy")
ax.legend()
ax.set_xlim(t[0], t[-1])
plt.savefig("sine_initial_guess_vs_true.pdf", bbox_inches="tight", format="pdf")
plt.show()

# 2. BEST FIT vs TRUE vs NOISE
fig, ax = plt.subplots(1, 1, figsize=(7,3), dpi=150)
ax.plot(t, noisy_data, color="k", label="Noisy data")
ax.plot(t, true_data, color="g", label="True signal")
ax.plot(t, best_fit_model, color="dodgerblue", label="Best-fit model")
ax.set_xlabel("Time [s]")
ax.set_ylabel("Signal Amplitude")
ax.set_title("Best Fit vs True vs Noisy")
ax.legend()
ax.set_xlim(t[0], t[-1])
plt.savefig("sine_best_fit_vs_true.pdf", bbox_inches="tight", format="pdf")
plt.show()
```


Observed data
Best-fit model
Signal Amplitude (arb. units)
Time [s]
6
4
2
0
−2
−4
−6
0
25
50
75
100
125
150
175
200


Observed data
True signal
Signal Amplitude (arb. units)
Time [s]
6
4
2
0
−2
−4
−6
0
25
50
75
100
125
150
175
200

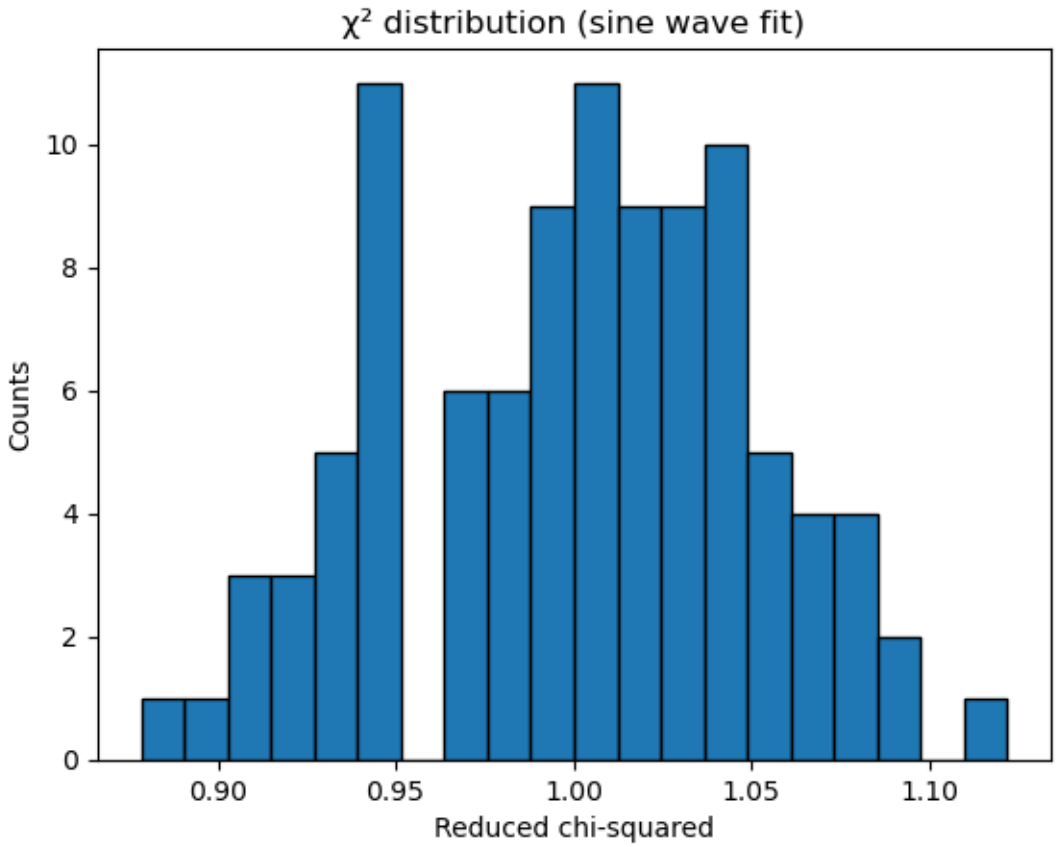




[140]:
```
sigma = 0.5
residuals = noisy_data - best_fit_model
chi2 = np.sum((residuals / sigma) ** 2)
dof = len(t) - 2
print("Reduced chi-squared:", chi2 / dof)
print("Iterations (nit):", results.nit)
```

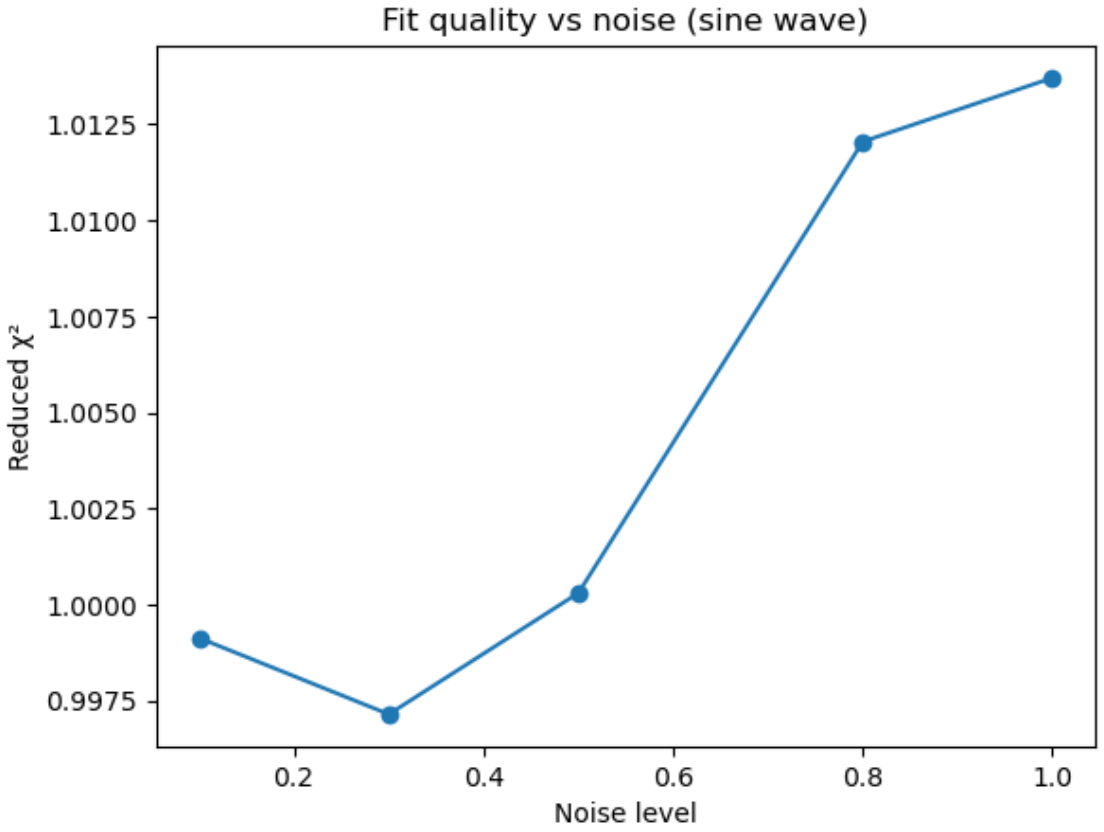
```
Reduced chi-squared: 1.0120618299345014
Iterations (nit): 5
```

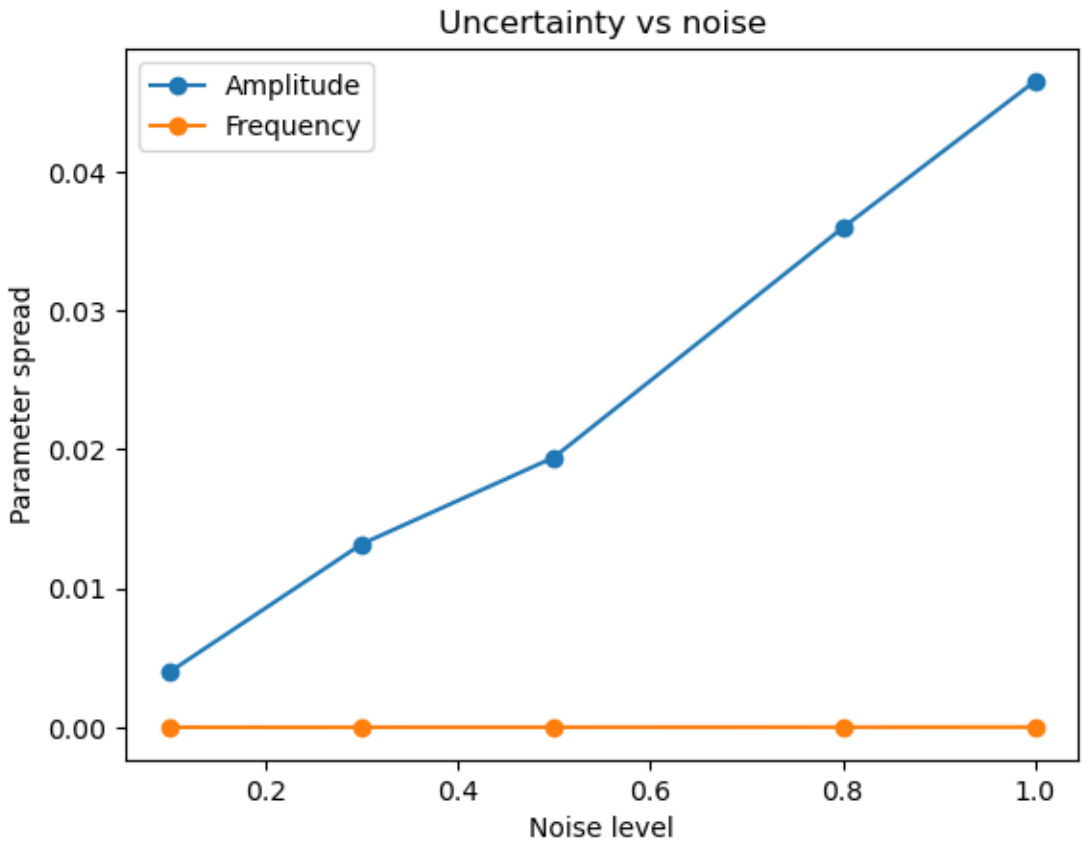
[120]:
```
N = 100
chi2_list = []
for i in range(N):
    noisy = true_data + np.random.normal(0, 0.5, size=t.size)
    res = minimize(
        fun=chisq,
        x0=init_guess,
        args=(noisy, t),
        jac=grad_chisq,
        method="BFGS"
    )

    fit = m(res.x[0], res.x[1], t)
    chi2 = np.sum(((noisy - fit) / 0.5) ** 2)
    chi2_list.append(chi2 / (len(t) - 2))

plt.hist(chi2_list, bins=20, edgecolor='k')
plt.xlabel("Reduced chi-squared")
plt.ylabel("Counts")
plt.title(" ² distribution (sine wave fit)")
plt.show()
```



[121]:
```
noise_levels = [0.1, 0.3, 0.5, 0.8, 1.0]

chi2_means = []
amp_spread = []
freq_spread = []

for sigma in noise_levels:

    chi2_vals = []
    A_vals = []
    nu_vals = []

    for i in range(50):

        noisy = true_data + np.random.normal(0, sigma, size=t.size)

        res = minimize(
            fun=chisq,
            x0=init_guess,
            args=(noisy, t),
            jac=grad_chisq,
            method="BFGS"
        )

        A_fit, nu_fit = res.x

        A_vals.append(A_fit)
        nu_vals.append(nu_fit)

        fit = m(A_fit, nu_fit, t)

        chi2 = np.sum(((noisy - fit) / sigma) ** 2)
        chi2_vals.append(chi2 / (len(t) - 2))

    chi2_means.append(np.mean(chi2_vals))
    amp_spread.append(np.std(A_vals))
    freq_spread.append(np.std(nu_vals))

plt.plot(noise_levels, chi2_means, marker='o')
plt.xlabel("Noise level")
plt.ylabel("Reduced  ²")
plt.title("Fit quality vs noise (sine wave)")
plt.show()

plt.plot(noise_levels, amp_spread, marker='o', label="Amplitude")
```

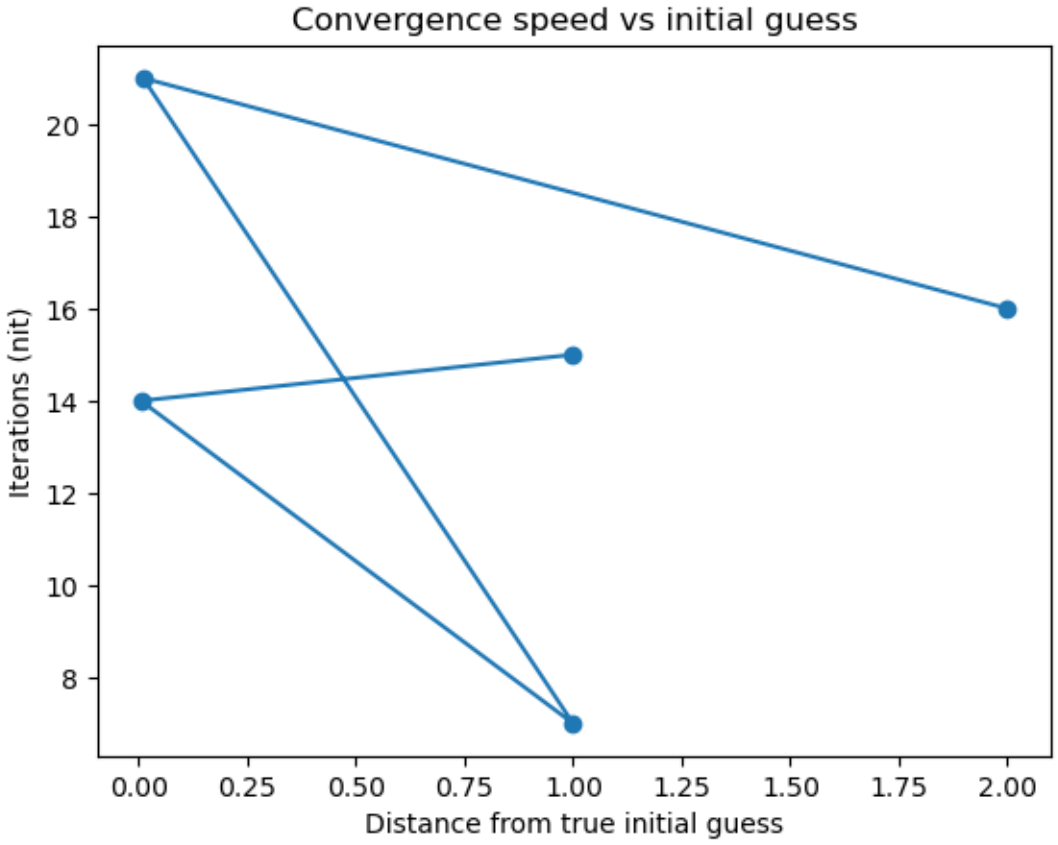
```
plt.plot(noise_levels, freq_spread, marker='o', label="Frequency")
plt.xlabel("Noise level")
plt.ylabel("Parameter spread")
plt.title("Uncertainty vs noise")
plt.legend()
plt.show()
```

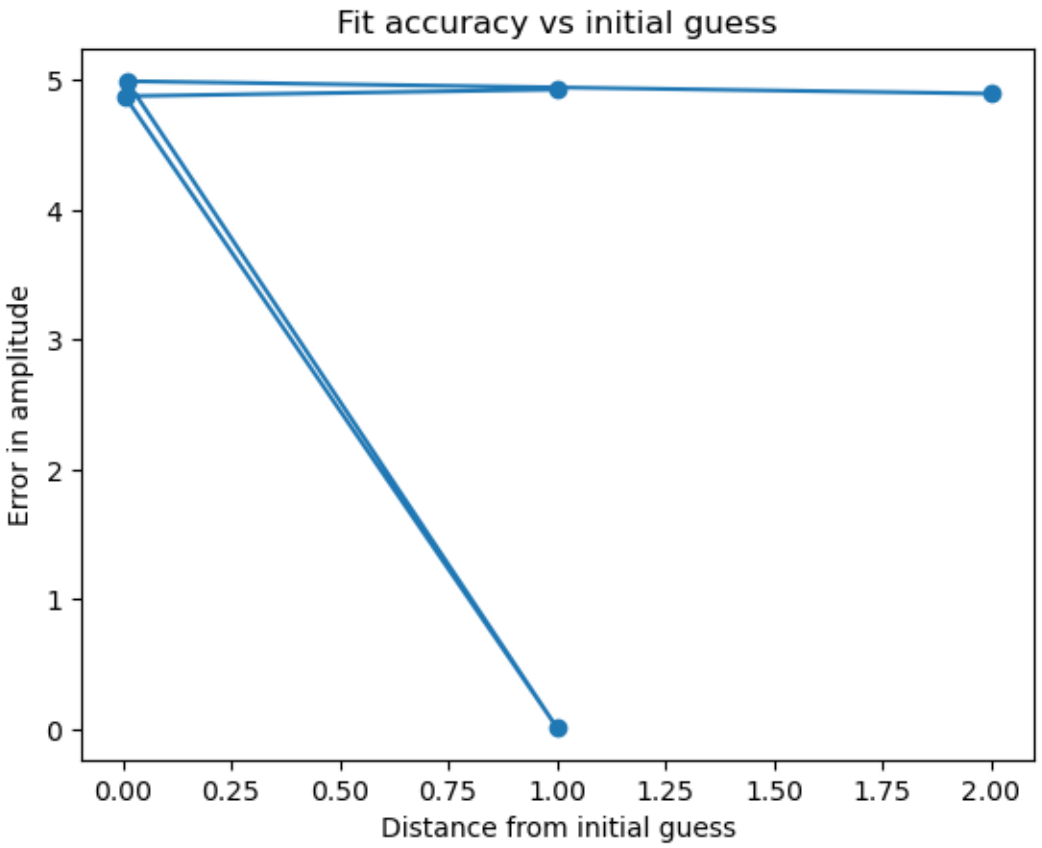




[122]:
```
init_guesses = [
    [4, 0.04],
    [5, 0.045],
    [6, 0.05],
    [5, 0.06],
    [7, 0.07]
]

iterations = []
A_error = []

for guess in init_guesses:

    res = minimize(
        fun=chisq,
        x0=np.array(guess),
        args=(noisy_data, t),
        jac=grad_chisq,
        method="BFGS"
    )
```

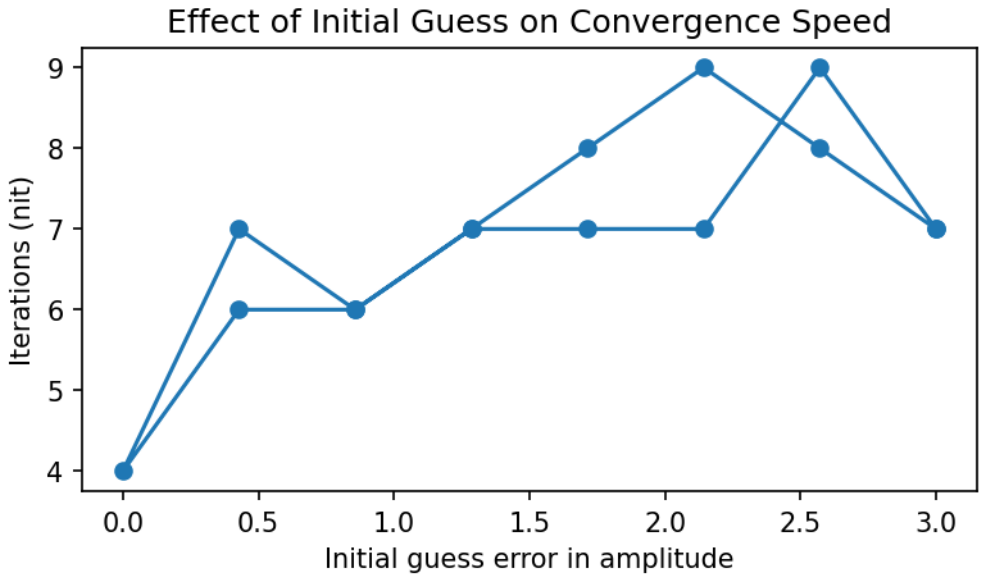
```
    iterations.append(res.nit)
    A_error.append(abs(res.x[0] - A0))

dist = [np.linalg.norm(np.array(g) - init_guess) for g in init_guesses]

plt.plot(dist, iterations, marker='o')
plt.xlabel("Distance from true initial guess")
plt.ylabel("Iterations (nit)")
plt.title("Convergence speed vs initial guess")
plt.show()

plt.plot(dist, A_error, marker='o')
plt.xlabel("Distance from initial guess")
plt.ylabel("Error in amplitude")
plt.title("Fit accuracy vs initial guess")
plt.show()
```

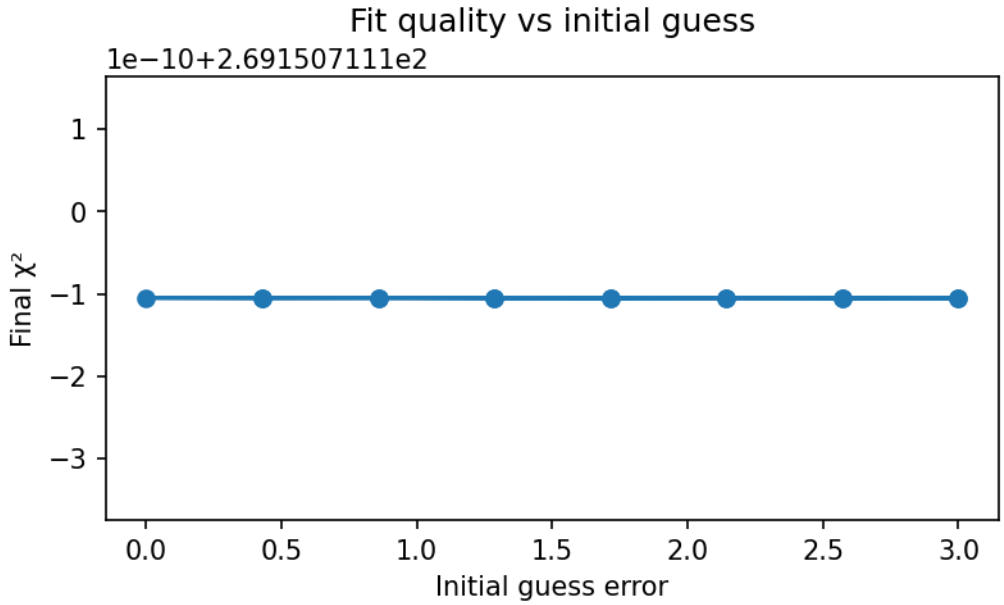




[66]:
```
from scipy.optimize import minimize
init_A_vals = np.linspace(2, 8, 15)
results_init = []
for A_guess in init_A_vals:
    init_guess = np.array([A_guess, 0.05])
    res = minimize(fun=chisq,x0=init_guess,args=(noisy_data,␣
 ↪t),jac=grad_chisq,method="BFGS")
    results_init.append([A_guess, res.x[0], res.x[1], res.nit, res.fun])
init_A = np.array([r[0] for r in results_init])
nit = np.array([r[3] for r in results_init])
error = np.abs(init_A - A0)
plt.figure(figsize=(6,3), dpi=150)
plt.plot(error, nit, marker='o')
plt.xlabel("Initial guess error in amplitude")
plt.ylabel("Iterations (nit)")
plt.title("Effect of Initial Guess on Convergence Speed")
plt.show()
chi2_vals = np.array([r[4] for r in results_init])
plt.figure(figsize=(6,3), dpi=150)
plt.plot(error, chi2_vals, marker='o')
```

```
plt.xlabel("Initial guess error")
plt.ylabel("Final  ²")
plt.title("Fit quality vs initial guess")
plt.show()
```





Does the best-fit provide a good description of the noisy data? If not, then something probably went wrong, and you should try debugging your code to fix it.

3 Maximum Likelihood

We can make these optimization problems a bit more well-rooted by framing them as a statistics problem:

How likely is it that the model describes the data I have?

In this case, if we think we understand the statistics of our data, then we can define a *likelihood* $\mathcal{L}$ and fidget with the model parameters until we have maximized the likelihood that the model describes the data we have been given. In a Bayesian scheme, usually the likelihood is written down as

$$\mathcal{L}(\theta|d),$$

which you can think of as quantifying "how likely is this set of model parameters θ given some (fixed) data d". (This is not the same as "how probable", but it can be used as a measure of how well the data is described by the model.)

Usually, we assume that the random fluctuations in our data are Gaussian, and this means that the likelihood is a joint Gaussian distribution:

$$\mathcal{L}(\theta|d) = \prod_i (2\pi\sigma_i^2)^{-1/2} \exp\left(-\frac{[d_i - m_i(\theta)]^2}{2\sigma_i^2}\right).$$

If we have N data points with equal variance, then this can be rewritten as

$$\mathcal{L}(\theta|d) = (2\pi\sigma^2)^{-N/2} \exp\left(-\frac{1}{2}\chi^2\right).$$

In this case, maximizing the likelihood is equivalent to minimizing χ^2, so we can employ the same tricks from the previous sections. This is a bit of a nicer extension though, since in a Bayesian framework we can also apply constraints based on *priors*, or rather leverage information about what values we think the model parameters ought to be able to take on (either from other experiments or from physical arguments).

I'm not going to say much more about this, because frankly I don't really think I'm qualified to say much more. Here are a few things I would like you to keep in mind, though:
* Maximizing the likelihood is equivalent to minimizing the negative log-likelihood. We have plenty of helpful tools available for minimizing things, and log-likelihoods are typically much better behaved numerically than likelihoods (exponentials are sensitive to small changes, and this can make numerical things involving exponentials difficult to work with). * CorrCal is formally framed as a maximum likelihood routine where the data is treated as *correlated* noise, and rather than model the data directly, we model the *covariance* in the data. This is a very non-traditional approach, but it's talked about in a bit more detail in the CorrCal tutorial notebook and paper.